

PSN10 : Conception d'un microcontrôleur simple en VHDL, basé sur une architecture RISC-V 32 bits

Contexte pédagogique

Le but du projet PSN10 est de développer vos compétences en systèmes numériques (processeurs, circuits numériques, ...), dans le cadre de la démarche compétence. Plus particulièrement, ce projet se focalisera sur les compétences (et les critères d'exigences associés) suivants :

- Compétence 1 : Analyser un système ou un besoin
- Compétence 2 : Concevoir un système cyber-physique
- Compétence 3 : Développer un système cyber-physique
- Compétence 4 : Caractériser un système cyber-physique

De plus, le but du projet est aussi de faire le lien entre différentes matières enseignées cette année :

- « Architecture des processeurs » (SN330) : Fonctionnement d'un processeur RISC-V et développement en langage assembleur
- « Introduction à la conception des circuits numériques » (SN360) : Conception de circuits numériques sur composants reprogrammables (FPGA)
- Projet « Algorithmique et programmation » (IN330) : Conception d'un assembleur et d'un émulateur pour un processeur basé sur un sous-ensemble du jeu d'instructions RISC-V 64 bits.

Enfin, ce projet vous introduira à la notion de microcontrôleurs, que vous approfondirez l'année prochaine, dans le cours « Programmation embarquée sur microcontrôleur » (SN420). Plus particulièrement, un microcontrôleur est un composant omniprésent dans les systèmes embarqués, il est généralement composé d'un processeur, de mémoires, d'entrées/sorties et de périphériques.

Présentation du projet

Vous souhaitez postuler en tant qu'ingénieur concepteur en VHDL chez ESISC-V, une entreprise Valentinoise spécialisée dans la conception de microcontrôleurs basé sur le jeu d'instructions RISC-V et fondée par d'anciens étudiants de Grenoble INP – Esisar. Lors de votre premier entretien d'embauche, on vous a laissé 2 jours pour concevoir et développer un prototype de microcontrôleur, basée sur un sous-ensemble du jeu d'instructions RV32I. A la fin de cette période, vous aurez une présentation de 5-10 minutes à faire pour présenter votre travail et faire une démonstration.

Pour cela, l'entreprise vous a fourni plusieurs ressources :

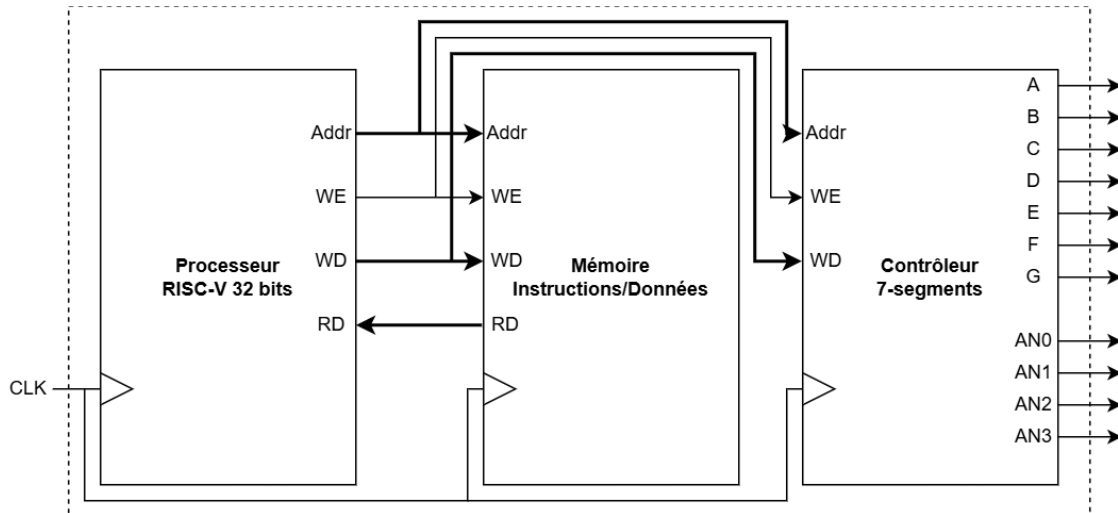
- L'équipe « Architecture » vous a fourni l'architecture du microcontrôleur, ainsi que le sous-ensemble du jeu d'instructions et la microarchitecture du processeur RISC-V 32 bits.
- L'équipe « Conception » vous a fourni des codes VHDL de blocs génériques de la microarchitecture (registres, multiplexeur, additionneurs, banc de registres, mémoires), ainsi que d'un périphérique permettant d'interagir avec un afficheur 4 x 7-segments.
- L'équipe « Validation » vous a fourni des *testbenchs* pour tester les différents éléments de l'architecture, allant des blocs génériques au microcontrôleur complet.
- L'équipe « Logicielle » vous a fourni deux exécutables permettant d'assembler et d'émuler un programme assembleur écrit pour ce jeu d'instructions, ainsi que des programmes de test.

PS : Il s'agit d'une situation fictive, aucune embauche n'est réellement en jeu dans ce projet.

Architecture du microcontrôleur

L'architecture du microcontrôleur que vous devez concevoir et développer sera composé des composants suivants :

- Un processeur, basé sur un sous-ensemble du jeu d'instructions RV32I
- Une mémoire contenant à la fois les instructions et les données
- Un périphérique permettant de contrôler un afficheur 4 x 7-segments



L'architecture du microcontrôleur simple à réaliser

Ce microcontrôleur sera déployé sur la carte de développement Basys 3 et interagira avec l'afficheur 4 x 7-segments présent sur la carte (en bas à gauche). Ainsi, le microcontrôleur pourra, par exemple, réaliser des calculs en interne et afficher le résultat sur l'afficheur 4 x 7-segments.

Remarque :

- Les fils en gras représente des bus de données, alors que les fils « normaux » représentent des bits uniques.
- La taille des bus de données est décrite plus loin dans le document.
- L'architecture ne comporte pas de signal de reset.

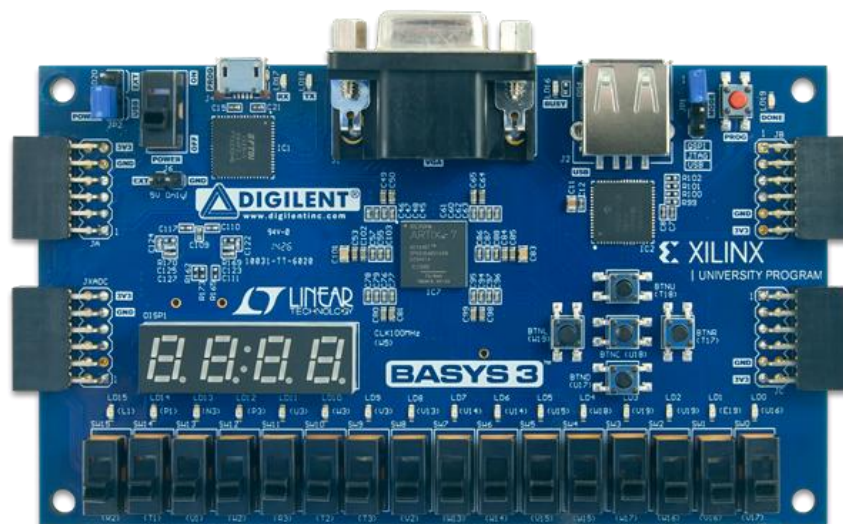


Photo vue du dessus de la carte de développement Basys 3 (Digilent.com)

Jeu d'instructions du processeur

Le jeu d'instruction (ou architecture externe du processeur) que vous implémenterez est une sous-partie du jeu d'instructions RV32I : *RISC-V 32 bits Base Integer Instructions*.

Pour rappel, un processeur RISC-V 32 bits est composé de 32 registres de 32 bits : x0 à x31. De plus, le registre x0 (zero) est un registre fixe, qui contient la valeur nulle.

Les instructions que vous implémenterez sont présentées dans le tableau ci-dessous :

Inst	Syntax	Format	Opcode	funct3	funct7	Description (C)
add	add rd, rd1, rd2	R	0110011	0x0	0x00	rd = rs1 + rs2
sub	sub rd, rd1, rd2	R	0110011	0x0	0x20	rd = rs1 - rs2
addi	addi rd, rd1, imm	I	0010011	0x0		rd = rs1 + imm
lw	lw rd, imm(rs1)	I	0000011	0x2		rd = M[rs1+imm]
sw	sw rs2, imm(rs1)	S	0100011	0x2		M[rs1 + imm] = rs2
beq	beq rd1, rd2, imm	B	1100011	0x0		if (rs1 == rs2) PC+= imm
bne	bne rd1, rd2, imm	B	1100011	0x1		if (rs1 != rs2) PC+= imm
blt	blt rd1, rd2, imm	B	1100011	0x4		if (rs1 < rs2) PC+= imm
bge	bge rd1, rd2, imm	B	1100011	0x5		if (rs1 >= rs2) PC+= imm
jal	jal rd, imm	J	1101111			rd = PC+4 ; PC += imm

Description du jeu d'instructions RISC-V 32 bits réduit

Microarchitecture du processeur

La microarchitecture (ou architecture interne) du processeur qui vous a été fourni est présentée dans la figure ci-dessous. Il s'agit d'une microarchitecture multi-cycle, i.e une instruction est exécutée en plusieurs cycles d'horloge. Dans notre cas, les différents cycles seront les suivants :

- *Fetch* : L'instruction est récupérée en mémoire et stockée dans le registre d'instruction (*Instr*)
- *Decode* : L'instruction est décodée et les registres concernés sont accédés
- *Execute* : L'instruction est exécutée (calculs de l'ALU, ...)
- *Memory access* : Les éventuelles données sont stockées (*sw*) ou récupérées (*lw*) en mémoire
- *Write-back* : Le résultat de l'instruction est stocké dans les registres et le *Program Counter* (PC) est incrémenté

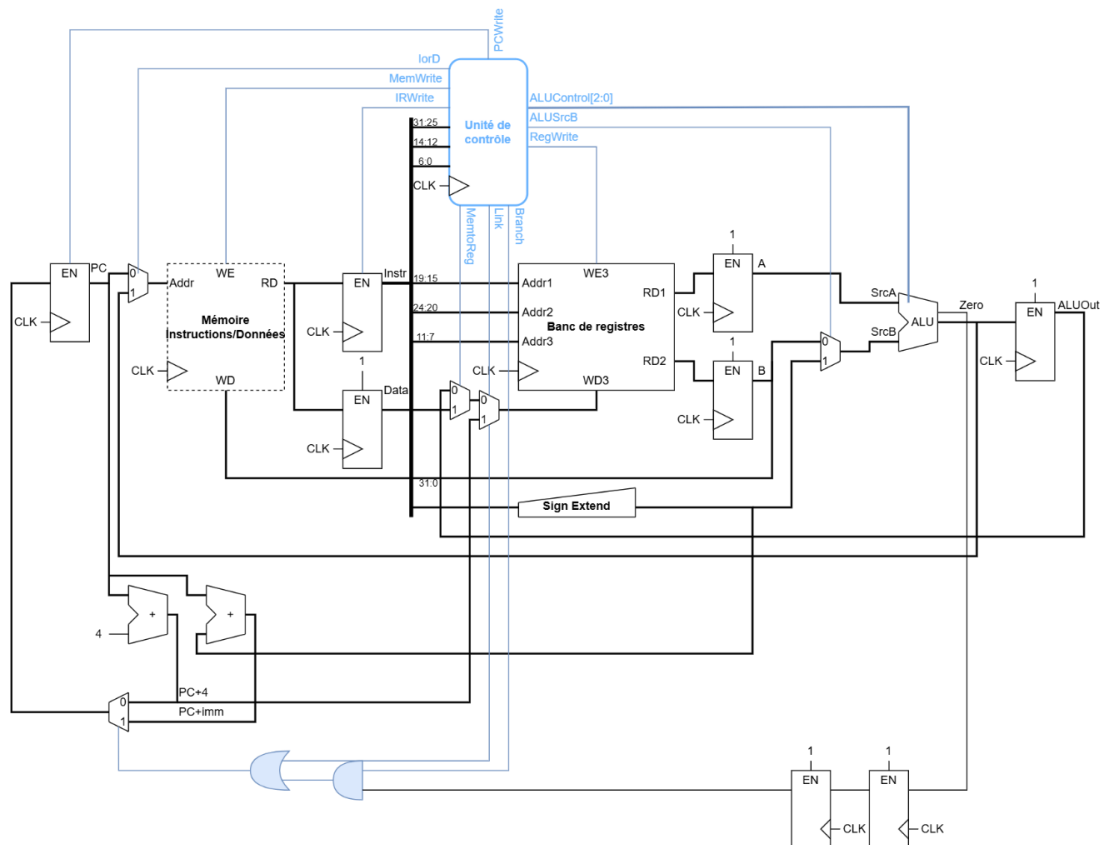
L'équipe « Conception » vous a fourni le code VHDL de blocs génériques qu'ils ont déjà développés pour d'autres projets :

- **Register32bits.vhd** : Le code VHDL d'un registre 32 bits, avec une entrée *clk* pour l'horloge, une entrée *enable* de contrôle et une entrée *input* de 32 bits et une sortie *output* de 32 bits.
- **MUX2x32bits.vhd** : Le code VHDL d'un multiplexeur (MUX) avec deux entrées *input0* et *input1* de 32 bits, une entrée de sélection *sel* et une sortie *output* de 32 bits.
- **Adder32bits.vhd** : Le code VHDL d'un additionneur de 32 bits, avec deux entrées *input0* et *input1* de 32 bits et une sortie *output* de 32 bits.
- **RegisterFile32x32bits.vhd** : Le code VHDL d'un banc de 32 registres de 32 bits (les registres manipulables par le processeur), avec une entrée *clk* pour l'horloge, trois entrées d'adresse *Addr1*, *Addr2* et *Addr3* de 5 bits, une entrée de contrôle d'écriture *WE3*, une entrée d'écriture de 32 bits *WD3* et deux sorties *RD1* et *RD2* de 32 bits.
- **Memory32bits.vhd** : Le code VHDL d'une mémoire permettant de stocker ou les lire les instructions et les données, avec une entrée *clk* pour l'horloge, une entrée *Addr* de 32 bits, une entrée de contrôle d'écriture *WE*, une entrée d'écriture *WD* de 32 bits et d'une sortie *RD* de 32 bits.

De plus, ils vous ont fourni un code VHDL **SevenSegmentPeripheral.vhd**, pour le contrôleur d'afficheur 4 x 7-segments, avec une entrée *clk* pour l'horloge, une entrée de *Addr* de 32 bits pour l'adresse du périphérique, une entrée *WD* de 32 bits pour la valeur à afficher, une entrée *WE* de contrôle d'écriture et des sorties de *A* à *G* et *AN0* à *AN3* pour contrôler l'afficheur 4 x 7-segments.

Remarques :

- Si la taille de certaines entrées/sorties n'est pas mentionnées, elle est implicitement de 1 bit pour les fils « normaux » et de 32 bits pour les fils gras.
- La lecture des données dans **FileRegister32x32bits** et **Memory32bits** est asynchrone, alors que l'écriture est synchrone.
- Les fils bleus correspondent à des signaux de contrôle.



La microarchitecture du processeur RISC-V 32 bits à implémenter

Ainsi, vous devrez tout d'abord concevoir les trois blocs de la microarchitecture qui sont spécifiques au jeu d'instructions choisi par l'équipe « Architecture » :

- **ControlUnit.vhd** : Le code VHDL de l'unité de contrôle permettant de piloter les différents éléments du processeur, avec une entrée *clk* pour l'horloge, une entrée *opcode* sur 7 bits, une entrée *funct3* sur 3 bits, une entrée *funct7* sur 7 bits, une sortie *IoD* pour le contrôle de l'origine de l'adresse, une sortie *MemWrite* pour le contrôle de l'écriture de la mémoire, une sortie *IRWrite* pour le contrôle de l'écriture du registre d'instruction, une sortie *MemtoReg* pour le contrôle de l'origine du registre en écriture, une sortie *Link* pour indiquer s'il s'agit d'une instruction *jal*, une sortie *Branch* pour indiquer s'il s'agit d'une instruction de branchement, une sortie *RegWrite* pour le contrôle de l'écriture des registres, une sortie *ALUSrcB* pour le contrôle de l'origine de *SrcB*, une sortie *ALUControl* sur 3 bits pour le contrôle de l'ALU, une sortie *PCWrite* pour le contrôle de l'écriture de PC.

- **ALU32bits.vhd** : Le code VHDL de l'unité arithmétique et logique (ALU) de 32 bits permettant de réaliser des différentes opérations arithmétiques et logiques, avec une entrée *mode* de 3 bits, deux entrées *SrcA* et *SrcB* de 32 bits, une sortie *zero (flag)* et une sortie *output* de 32 bits.
- **SignExtend.vhd** : Le code VHDL de l'unité d'extension de signe de 32 bits permettant d'étendre les valeurs immédiates contenues dans les instructions sur 32 bits, avec une entrée *instruct* de 32 bits et une sortie *imm* de 32 bits.

De plus, il faudra concevoir **ProcessorRISCV32bits.vhd**, le code VHDL de la microarchitecture du processeur RISC-V 32 bits (sans la mémoire), avec une entrée *clk* pour l'horloge, une entrée *RD* de 32 bits pour la lecture depuis la mémoire ou les périphériques, une sortie *WE* pour le contrôle de l'écriture, une sortie *Addr* de 32 bits pour la sélection de l'adresse mémoire ou des périphériques, une sortie *WD* de 32 bits pour l'écriture vers la mémoire ou les périphériques.

Enfin, vous devrez aussi concevoir **MicrocontrollerRV32bits.vhd**, le code VHDL de l'architecture du microcontrôleur, avec une entrée *clk* pour l'horloge et les sorties *A* à *G* et *ANO* à *AN3* pour contrôler l'afficheur 4 x 7-segments.

Validation des blocs matériels

De son côté, l'équipe « Validation », normalement en charge de la validation des codes VHDL produits par l'équipe « Conception », vous a fourni différents *testbenchs* pour comprendre le fonctionnement des blocs fournis, ainsi que pour valider les blocs que vous devez développer :

- **testbenchRegister32bits.vhd**, permettant de valider le bon fonctionnement des registres 32 bits (**Register32bits.vhd**).
- **testbenchMUX2x32bits.vhd**, permettant de valider le bon fonctionnement des multiplexeurs 2 x 32 bits (**MUX2x32bits.vhd**).
- **testbenchAdder32bits.vhd**, permettant de valider le bon fonctionnement des additionneurs 32 bits (**Adder32bits.vhd**).
- **testbenchRegisterFile32x32bits.vhd**, permettant de valider le bon fonctionnement du banc de 32 registres de 32 bits (**RegisterFile32x32bits.vhd**).
- **testbenchMemory32bits.vhd**, permettant de valider le bon fonctionnement de la mémoire contenant les instructions et les données (**Memory32bits.vhd**).
- **testbenchSevenSegmentsPeripheral.vhd**, permettant de valider le bon fonctionnement du contrôleur de l'afficheur 4 x 7-segments (**SevenSegmentsPeripheral.vhd**).
- **testbenchControlUnit.vhd**, permettant de valider le bon fonctionnement de l'unité de contrôle (**ControlUnit.vhd**).
- **testbenchALU32bits.vhd**, permettant de valider le bon fonctionnement de l'unité arithmétique et logique (**ALU32bits.vhd**).
- **testbenchSignExtend.vhd**, permettant de valider le bon fonctionnement de l'unité d'extension de signe (**SignExtend.vhd**).
- **testbenchProcessorRISCV32bits.vhd**, permettant de valider le bon fonctionnement du processeur (**ProcessorRISCV32bits.vhd**).
- **testbenchMicrocontrollerRV32bits.vhd**, permettant de valider le bon fonctionnement du microcontrôleur (**MicrocontrollerRV32bits.vhd**).

Vous utiliserez ainsi les différents *testbenchs*, pour valider le fonctionnement des blocs (fournis ou produits). L'étape de validation finale consistera à valider le bon fonctionnement du processeur directement sur la Basys 3.

Assembleur et émulateur

Pour le développement de programmes spécifiques à votre application, qui ne seraient pas couverts par les *testbenchs* fournis par l'équipe « Validation », l'équipe « Logiciel » vous a fourni deux exécutables permettant d'assembler et d'émuler des programmes écrits avec le sous-ensemble du jeu d'instructions RV32I (fonctionnement uniquement sous les Linux des salles de TPs) :

- **riscv-assembler** : Un exécutable permettant de convertir un programme en assembleur (.s) en un fichier contenant le code machine en hexadécimal (.hex).
- **riscv-emulator** : Un exécutable permettant d'émuler le fonctionnement du processeur à partir du code machine en hexadécimal (.hex) et de renvoyer l'état du processeur à la fin de l'exécution (.state).

L'équipe vous a aussi fournis plusieurs programmes de tests (.s) qui ont servis à l'équipe « Validation » pour réaliser les différents *testbenchs*, ainsi les codes assemblés (.hex) et les résultats attendus (.state).

Remarques :

- Les pseudo-instructions suivants sont supportées par **riscv-assembler** :
 - j offset: jal x0, offset
 - mv rd, rs1: addi rd, rs1, imm
 - li rd, imm: addi rd, zero, imm
- L'exécutable **riscv-assembler** fonctionne de la manière suivante :

./riscv-assembler /path/to/source.s /path/to/destination.hex

- L'exécutable **riscv-emulator** fonctionne de la manière suivante :

./riscv-emulator /path/to/source.hex /path/to/destination.state

Éléments fournis

L'archive « ESISCV-Microcontroller-Hiring.zip » fournie par l'entreprise, contient :

- /conception : Un dossier contenant les différents blocs VHDL fournis
- /validation : Un dossier contenant les différents *testbenchs*
- /logiciel : Un dossier contenant les deux exécutables et des programmes de tests
- Basys-3-Master.xdc : Le fichier de contraintes

Pour aller plus loin

Si vous le souhaitez, vous pouvez continuer le projet, que ce soit en améliorant la microarchitecture, en créant de nouveaux périphériques ou encore en créant d'autres programmes assembleurs.

Modalité d'évaluation

En plus de l'attestation de compétence que vous devez remplir, vous serez évalué 25% sur votre implication en séance, 50% sur votre développement et 25% sur une présentation de 5-10 minutes.

Références

- [1] SN330 : Architecture des processeurs – D. Hély ([Chamilo](#))
- [2] SN360 : Introduction à la conception des circuits numériques – V. Beroulle ([Chamilo](#))
- [3] Computer organization and design – D. A. Patterson et al. ([Lien](#))
- [4] Single-cycle vs multi-cycle processors – E. Harman ([Medium](#))